

SprintAI: Deep System Architecture, Codebase Internals & Staff Interview Guide

An exhaustive technical walkthrough of a production-grade AI study and exam platform using Next.js 16, Redis, BullMQ, PostgreSQL, pgvector, and Google Gemini.

Author: Anubhav
Goel

System: SprintAI (Full-Stack Distributed
Architecture)

Target Level: Full-Stack / Distributed Systems / AI
Engineer

1. Executive Summary & Core Value Proposition

SprintAI is a distributed, full-stack, AI-powered study platform designed to solve a fundamental learning bottleneck: studying massive, complex technical textbooks, lecture slide decks, and scientific documentation is notoriously slow and passive. SprintAI transforms unstructured PDF documents into an active, masterclass-level learning environment consisting of:

- **Comprehensive, Masterclass Study Notes:** Structured textbook-quality notes rendered with full LaTeX mathematical typesetting (via KaTeX).
- **Rigorous 10-Question Interactive Quizzes:** Auto-generated with Zod schema validation, instant grading, and in-depth conceptual explanations for every option.
- **High-Yield Flashcards:** Key definitions and formulas with 3D flip card animations.
- **Contextual RAG Retrieval Q&A:** Instant semantic questioning grounded purely in the uploaded document using PostgreSQL `pgvector` similarity search.

The Core Engineering Challenge Solved

In serverless hosting environments like Vercel, serverless function executions terminate after sending a response or freeze after a few seconds. Extracting text from 50-page PDFs, generating sliding-window text chunks, creating high-dimensional vector embeddings, and running dual multi-thousand-token LLM generations can take **10 to 25 seconds**.

Running this synchronously inside an HTTP route causes catastrophic HTTP 504 Gateway Timeouts. SprintAI solves this by implementing a **decoupled, distributed producer-consumer queue architecture** using **Upstash Redis** and a persistent **BullMQ Worker hosted on Render** with zero cold-start latency.

2. High-Level Distributed Architecture Diagram



3. Technology Stack Matrix & Library Justifications

Layer / Category	Package / Tool	Version	Architectural Role & Why We Chose It
Frontend Framework	Next.js (App Router)	16.3.3	React Server Components (RSC) by default for zero client JS overhead, App Router nested layouts, Server Actions, route handlers.
UI Library & State	React	19.2.8	Latest React concurrency, React actions, useOptimistic, server-side data streaming.
Styling & Design	Tailwind CSS v4 + shadcn/ui	^4.3.3	Utility-first CSS, high performance CSS engine with zero build-time configuration, accessible component primitives via Radix / Base UI.
Authentication	Better Auth	^1.7.2	TypeScript-first authentication framework. Direct database session management via Prisma adapter, native Google OAuth, secure HTTP-only cookies.
ORM & Database Client	Prisma ORM + PostgreSQL	^7.10.0	Type-safe database modeling, schema migrations, and connection pooling with raw SQL vector extensions.
Vector Search Engine	pgvector	Postgres Extension	Embeds semantic search directly into the relational database using the cosine distance operator ($\leftrightarrow$). Eliminates the cost of standalone vector databases.
Message Broker / Queue	Upstash Redis + BullMQ	ioredis ^6.0.0 bullmq ^6.3.1	Robust background task queue with automated retries, exponential backoff, atomic job locking, and memory-efficient distributed processing.
Document Ingestion	pdf-parse	^2.4.5	In-memory Node.js binary PDF parser. Extracts full text without needing system binaries like Poppler or pdftotext.
AI Intelligence & Embeddings	@google/genai	^2.19.0	gemini-3.6-flash for high-speed multi-token generation; gemini-embedding-001 for 768-dimensional semantic vector embeddings.
Mathematical Rendering	KaTeX + remark-math + rehype-katex	^0.18.7 ^6.0.0 ^7.0.1	Parses inline ($\$ \dots \$$) and block ($\$ \$ \dots \$ \$$) LaTeX formulas into accessible, perfectly typeset math HTML on the client.
Object Storage	Cloudinary	^2.11.0	Permanent, CDN-backed PDF asset storage. Prevents bloat in PostgreSQL and Redis.

4. End-to-End System Lifecycles (Step-by-Step)

Phase 1: Bootstrapping & User Authentication

SprintAI uses **Better Auth** with a PostgreSQL database adapter managed by Prisma.

1. **User Navigation:** User hits `/signin` or `/signup`.
2. **Provider Initiation:** Clicking "Continue with Google" triggers `authClient.signIn.social({ provider: 'google' })`, redirecting the user to Google's OAuth 2.0 consent screen.
3. **Callback & Verification:** Google redirects to `/api/auth/callback/google` handled by the catch-all Better Auth handler `src/app/api/auth/[...all]/route.ts`.
4. **Session Persistence:** Better Auth creates a record in the `User` table (if new) and creates a cryptographically signed session in the `Session` table. It issues a secure, `HttpOnly`, `SameSite=Lax` cookie to the browser.
5. **Session Verification:** Any protected route or API handler (such as `/api/upload`) calls:

```
const session = await auth.api.getSession({ headers: await headers() });
if (!session) return Response.json({ error: 'Unauthorized' }, { status: 401 });
```

Phase 2: Document Upload & The Fast Ingestion Handshake

To prevent user frustration and serverless timeouts, the upload route **never** does heavy processing synchronously. It operates purely as a fast dispatcher.

Top-to-Bottom Execution Flow of `src/app/api/upload/route.ts`

1. **Session Authentication:** Verifies the user's session token from request headers.
2. **Multipart Form Parsing:** Extracts the `file` (PDF) and the initial `question`.
3. **Cloud Storage Offloading:** Converts the file into an in-memory buffer and uploads it to Cloudinary using `uploadToCloudinary(buffer, file.name)`. Cloudinary returns a secure HTTPS URL (`fileUrl`).
4. **Database Record Creation:**
 - Inserts a row into `Document` with `status = 'PROCESSING'`, `fileUrl`, and `userId`.
 - Creates a `Conversation` linked to the `Document` and `userId`.
 - Saves the user's initial question into `Message` with `isUser = true`.

5. Queueing the Lightweight Job Ticket:

Instead of passing massive PDF bytes through Redis (which would exhaust Redis RAM), it passes only the essential job metadata:

```
await documentQueue.add(
  'process-document',
  { documentId: document.id, fileUrl },
  {
    attempts: 3,
    backoff: { type: 'exponential', delay: 3000 },
    removeOnComplete: true,
  }
);
```

6. **Instant Return:** Returns `{ documentId: document.id }` with status `201 Created` in ~1 second.
7. **Client Navigation:** The frontend router instantly redirects the browser to `/notes/[documentId]/notes`.

Phase 3: The Background BullMQ Worker Pipeline

On Render, the standalone worker process (`src/workers/document.worker.ts`) runs continuously via `npm run worker` . It executes the following 7-step deterministic pipeline:

Step 1: PDF Download

The worker receives the job ticket containing `{ documentId, fileUrl }` . It fetches the binary buffer from Cloudinary via native `fetch(fileUrl)` and converts it to a Node `Buffer` .

Step 2: Text Extraction via `pdf-parse`

Runs `PDFParse({ data: buffer })` to extract text from all PDF pages in memory. A defensive `DOMMatrix` polyfill is defined on `globalThis` to ensure compatibility across Node.js runtime environments.

Step 3: Sliding-Window Text Chunking

The extracted raw text is passed to `chunkText()` (in `src/services/chunker.service.ts`):

- Splits text into words by whitespace.
- Groups words into chunks of **600 words** each.
- Applies a **50-word overlap** between consecutive chunks.

Why this matters: Overlap prevents critical context, formulas, or sentences from being severed at chunk boundaries, ensuring high semantic recall during vector search.

Step 4: Embedding Generation & PostgreSQL pgvector Storage

To ensure idempotency in case of job retries, the worker first deletes any stale chunks or quizzes for this document ID:

```
await prisma.documentChunk.deleteMany({ where: { documentId } });
await prisma.quiz.deleteMany({ where: { documentId } });
```

For each chunk, it calls Google's `gemini-embedding-001` model configured with `outputDimensionality: 768` . It creates a `DocumentChunk` record, then writes the vector embedding using parameterized raw SQL:

```
const vectorString = `[${embedding.join(',')}]`;
await prisma.$executeRawUnsafe(
  `UPDATE document_chunk SET embedding = $1::vector WHERE id = $2`,
  vectorString,
  chunk.id
);
```

Step 5: Parallel Dual-Track AI Generation

Study notes and interactive quizzes are completely independent. To cut latency in half, the worker invokes both concurrently using `Promise.all` :

```
const [notesContent, questions] = await Promise.all([
  generateStudyNotes(chunks),
  generateQuiz(chunks),
]);
```

- **Study Notes Prompt:** Uses `NOTES_SYSTEM_PROMPT` to force textbook-quality markdown, key intuition, mathematical formulas, trade-offs, and terminology tables.
- **Quiz Prompt:** Uses `QUIZ_SYSTEM_PROMPT` with structured JSON output enforced by Gemini's `responseSchema` and validated by Zod against `QuizQuestionSchema` .
- **Resilient Model Fallback:** Calls `gemini-3.6-flash` first. If a 503 or overload occurs, it catches the error and retries with `gemini-2.5-flash` .

Step 6 & 7: Atomic Database Commit & State Transition

Saves the parsed quiz questions into the `Quiz` table, updates `Document` with `notesContent` , and flips `status = 'READY'` .

Phase 4: Frontend Hydration, Real-Time Polling & KaTeX Math

1. **Client Polling:** While the document is in `PROCESSING` state, `NotesContent.tsx` mounts a 3-second interval polling `/api/notes/[id]`.
2. **State Transition:** Once the status updates to `READY`, the loading skeleton unmounts, and the markdown content is passed to `<ReactMarkdown>`.
3. **Mathematical Formula Typesetting:**
Raw text output like `$X = \{x_1, \dots, x_n\}$` is intercepted by `remark-math` and transformed by `rehype-katex` into clean KaTeX DOM nodes styled with `katex/dist/katex.min.css`:

```
<ReactMarkdown
  remarkPlugins={[remarkGfm, remarkMath]}
  rehypePlugins={[rehypeKatex]}
  components={{ ... }}
>
  {notesContent}
</ReactMarkdown>
```

4. **Interactive Quiz Engine:** `QuizEngine.tsx` loads the questions from `/api/notes/[id]/quiz`, manages a client state machine (selected answer, feedback display, score calculation, option-by-option explanations).
5. **Interactive Flashcards:** `FlashcardsContent.tsx` provides interactive 3D flip card cards, allowing students to test memory recall.

Phase 5: Contextual RAG Semantic Q&A

When a user asks a follow-up question in the chat sidebar:

1. **Vectorization:** The question is vectorized using `gemini-embedding-001`.
2. **Cosine Similarity Retrieval:** PostgreSQL searches for the 15 closest chunks via pgvector:

```
SELECT content
FROM document_chunk
WHERE "documentId" = $2 AND embedding IS NOT NULL
ORDER BY embedding <=> $1::vector ASC
LIMIT 15;
```

3. **Grounded Generation:** The 15 retrieved chunks are formatted into a context window and sent to `gemini-3.6-flash` with `EXPLAIN_SYSTEM_PROMPT`, which strictly forbids hallucinations and forces answers grounded purely in document excerpts.

5. Script & Library Execution Matrix

This matrix gives you a quick mental map of which file runs where, what triggers it, and which libraries it depends on:

File / Script	Trigger / Event	Runtime Env	Key Libraries	Core Purpose
src/lib/auth.ts src/lib/auth-client.ts	App startup / User sign-in / Cookie validation	Vercel (Serverless) & Browser	better-auth , @prisma/client	Configures Google OAuth, email/password auth, session cookies, and client auth state.
src/app/api/upload/route.ts	User clicks "Upload" button	Vercel (Serverless HTTP)	cloudinary , bullmq , ioredis , @prisma/client	Uploads PDF to Cloudinary, creates initial DB records, and enqueues job ticket to Redis.
src/lib/redis.ts	Upload route & Worker boot	Vercel & Render Worker	ioredis , bullmq	Instantiates shared Redis connection with TLS and exports the document-queue .
src/workers/document.worker.ts	Runs 24/7 (npm run worker)	Render (Linux Web Service)	bullmq , pdf-parse , node:http , @google/genai , prisma	Downloads PDF, extracts text, chunks, embeds, generates notes & quiz, serves HTTP health check.
src/services/chunker.service.ts	Called by worker during ingestion	Render Worker	Pure TypeScript	Sliding-window chunking: 600 words per chunk with 50-word overlap.
src/services/gemini.service.ts	Called by worker & Q&A routes	Render Worker & Vercel Serverless	@google/genai , zod , prisma	Embeddings, Study Notes, Quiz (JSON validated), Flashcards, and RAG Q&A retrieval.
src/components/NotesContent.tsx	User navigates to /notes/[id]/notes	Client Browser (React 19)	react-markdown , remark-gfm , remark-math , rehype-katex , katex	Polls document status, renders notes markdown with formatted

File / Script	Trigger / Event	Runtime Env	Key Libraries	Core Purpose
				LaTeX formulas.
src/components/QuizEngine.tsx	User navigates to /notes/[id]/quiz	Client Browser (React 19)	React, Lucide icons, Tailwind v4	Interactive quiz state machine with scoring and option explanations.

6. Core Architectural Decisions & Staff-Level Rationale ("The Why")

Why Redis + BullMQ instead of Next.js `after()` ?

The Naive Approach: Next.js 16 introduced `after()`, which allows scheduled tasks to run after an HTTP response is sent. However, in serverless environments (Vercel Lambdas), the underlying container can freeze, sleep, or terminate within milliseconds once the response completes. If a 50-page PDF takes 20 seconds to process, the Lambda function is killed mid-way, leaving the document stuck in `PROCESSING` forever.

The Production Solution: Redis + BullMQ decouples the web tier from the processing tier. Vercel acts as a lightweight producer (1s response time), while an always-on Render worker processes tasks reliably with atomic Redis locks, automated exponential backoff retries, and persistence.

Why Separate Web (Vercel) and Worker (Render)?

Workload Segregation: Web requests require low latency, high concurrency, and global edge caching (which Vercel excels at). Heavy background tasks require sustained CPU, persistent socket connections to Redis, and long-running memory lifecycles without execution limits (which Render excels at). Splitting them prevents CPU-heavy PDF parsing from choking user-facing HTTP requests.

Why 600-Word Chunks with 50-Word Overlap?

Chunk Size Trade-Off: If chunks are too small (e.g., 50 words), semantic context is lost. If chunks are too large (e.g., 3,000 words), vector embeddings become diluted averages of too many topics, hurting cosine distance precision. 600 words (~800 tokens) perfectly captures an entire conceptual topic or theorem.

The Overlap Rationale: A 50-word sliding overlap guarantees that any sentence or equation that spans across a chunk boundary is fully preserved in at least one chunk.

Why PostgreSQL `pgvector` over Pinecone or Weaviate?

Data Consistency & ACID Transactions: With `pgvector`, vector embeddings live in the exact same database table (`document_chunk`) as the relational text content. When a document is deleted, cascading foreign keys automatically delete all corresponding embeddings. There is no risk of out-of-sync data or orphan vectors.

Cost & Operational Simplicity: Pinecone and Weaviate charge separate monthly platform fees and require extra network hops. `pgvector` runs directly inside your existing PostgreSQL instance at zero additional cost.

Why Pure Vector Search vs. Hybrid Search with RRF?

SprintAI currently uses dense semantic vector search via `pgvector` cosine distance ($\Leftrightarrow$). This is ideal for conceptual questions. If exact keyword matches (e.g., specific error codes, variable names, or theorem numbers) need to be boosted, the architecture is designed so that PostgreSQL full-text search (`to_tsvector`) can be combined with `pgvector` using **Reciprocal Rank Fusion (RRF)**:

$$\text{RRF Score}(d) = \sum [1 / (60 + \text{Rank}_{\text{search}}(d))]$$

7. The Staff-Level Interview Question & Answer Bank

Q1: Can you walk me through the end-to-end architecture of SprintAI?

"SprintAI is an asynchronous, event-driven AI platform built with a clear separation of concerns between web traffic and heavy processing. The frontend is Next.js 16 deployed on Vercel. When a user uploads a PDF, the upload route stores the file on Cloudinary, writes an initial record in Neon PostgreSQL, and enqueues a job ticket with BullMQ on Upstash Redis before immediately returning a 201 Created response in ~1.2s. A persistent Node.js worker hosted on Render picks up the job, parses the PDF into text with `pdf-parse`, chunks it using a 600-word sliding window with 50-word overlap, generates 768-dimensional embeddings via Google Gemini, and persists them into PostgreSQL using `pgvector`. It then executes parallel LLM calls to generate masterclass study notes and a Zod-validated 10-question quiz before marking the document ready. The client polls the status and renders the study notes with full LaTeX math typesetting via KaTeX."

Q2: Why did you migrate from Next.js `after()` to Redis and BullMQ?

"Next.js 16's `after()` is designed for secondary serverless tasks like logging or analytics that take 100-200ms. In contrast, processing a 50-page technical PDF involves network downloads, CPU-intensive parsing, chunking, up to 50 embedding API calls, and dual LLM generation calls that can take 15-20 seconds. In a serverless environment like Vercel, functions freeze or terminate when the HTTP response completes. This caused sporadic failures where documents were permanently stuck in a processing state. By moving to BullMQ and Redis, we achieved guaranteed at-least-once processing, isolated failure domains, automatic exponential backoff retries, and decoupled scaling between our web tier and background compute."

Q3: How do you keep your Render background worker alive on the free tier?

"Render's free tier allows Web Services, but spins them down after 15 minutes of inbound HTTP inactivity. However, free background workers are not supported. To solve this cost-effectively, I embedded a minimal 5-line native Node.js HTTP server (`node:http`) inside the worker process listening on `process.env.PORT`, returning a `200 OK` health check. I then configured a free external cron job via `cron-job.org` to ping this endpoint every 10 minutes. This prevents the container from ever idling or sleeping, ensuring our BullMQ worker stays hot 24/7 with zero spin-down latency."

Q4: How do you prevent duplicate database records if a BullMQ job fails halfway through and retries?

"Idempotency is built directly into the worker pipeline. If a job fails at Step 5 (AI generation) and BullMQ retries the job 3 seconds later, executing Step 4 again could create duplicate text chunks or violate the 1-to-1 unique constraint on the Quiz table. To prevent this, Step 4 begins with an idempotent cleanup phase: `await prisma.documentChunk.deleteMany({ where: { documentId } })`; and `await prisma.quiz.deleteMany({ where: { documentId } })`; . This guarantees that every retry begins with a clean slate for that document. Furthermore, the worker only sets the document status to `FAILED` inside the `worker.on('failed')` event listener when all retry attempts have been completely exhausted (`job.attemptsMade ≥ job.opts.attempts`)."

Q5: How does your RAG retrieval pipeline prevent hallucinations?

"Our RAG pipeline enforces strict grounding at three levels: First, retrieval quality: we use `gemini-embedding-001` to vectorize the user's question, and query PostgreSQL using pgvector's cosine distance operator (`⇔`) to retrieve the top 15 most semantically relevant 600-word chunks. Second, prompt engineering: our `EXPLAIN_SYSTEM_PROMPT` explicitly instructs the model: 'Answer the user's question using ONLY the provided document excerpts. Do not invent or hallucinate facts not present in the excerpts.' Third, low temperature: we set the LLM temperature to `0.2` for factual deterministic synthesis. If the answer does not exist in the retrieved excerpts, the model explicitly states that the document does not contain that information."

Q6: Why did you choose pgvector instead of a dedicated vector database like Pinecone?

"Operational simplicity, transaction integrity, and cost. With pgvector, vector embeddings reside in the `document_chunk` table in the same PostgreSQL database as documents, users, and quizzes. When a user deletes a document, relational foreign key cascading deletes all associated chunks and vectors in a single ACID transaction. Dedicated vector databases like Pinecone require maintaining two separate database connections, synchronizing deletes manually to prevent orphaned vectors, and paying an additional minimum monthly infrastructure fee."

Q7: How did you fix the raw LaTeX rendering issue in the frontend?

"When Google Gemini outputs mathematical equations, it formats them using standard LaTeX notation, such as `$x = \{x_1, \dots, x_n\}$` . Standard Markdown parsers render these as literal dollar signs and curly braces. To resolve this, I extended `ReactMarkdown` with `remark-math` as a Markdown AST parser, `rehype-katex` as an HTML transformer, and imported KaTeX's official CSS (`katex/dist/katex.min.css`). This converts inline and display math tokens directly into accessible, beautifully typeset KaTeX DOM structures on client render."

Q8: How did you handle Gemini API model deprecations and demand spikes (503 errors)?

"When Google deprecated `gemini-2.0-flash` and experienced temporary demand spikes (HTTP 503) on experimental models like `gemini-3.5-flash-lite`, I implemented a two-layer resiliency strategy: First, at the service layer, I established `gemini-3.6-flash` as the primary model and wrapped all API calls in try-catch blocks that automatically fall back to `gemini-2.5-flash` if the primary model encounters a 503 or overload. Second, at the queue layer, BullMQ is configured with 3 retry attempts and exponential backoff (`delay: 3000`), allowing transient network or API spikes to resolve before the job re-executes."

Q9: How would you scale this architecture to handle 1,000,000 active users?

"To scale to 1M users, I would address three key bottlenecks: 1. Worker Horizontal Scaling: Because BullMQ and Redis handle job locking atomically, we can scale our Render worker horizontally from 1 container to 20+ worker pods without changing a single line of application code. Workers will automatically pull jobs off the Redis queue concurrently. 2. Vector Indexing: As the `document_chunk` table grows into millions of rows, sequential vector scans slow down. We would add an HNSW (Hierarchical Navigable Small World) index: `CREATE INDEX ON document_chunk USING hnsw (embedding vector_cosine_ops)`, reducing query times from $O(N)$ to $O(\log N)$. 3. Database Read Replicas & Connection Pooling: Enable Neon serverless connection pooling (PgBouncer) to handle thousands of concurrent serverless frontend connections without exhausting database connection limits."

Q10: Tell me about a difficult bug you encountered during development and how you resolved it.

"A notable bug occurred during local testing of the worker script on Windows. Node.js failed with an `ELF: SyntaxError` when running `tsx` because the project dependencies had originally been installed in a Linux WSL environment, creating Linux symlinks in `node_modules/.bin` and installing `@esbuild/linux-x64` instead of the Windows binary. Rather than corrupting the cross-platform environment, I recognized that in production on Render (a pure Linux container), the Linux binaries would execute natively. I verified TypeScript compilation using `node ./node_modules/typescript/bin/tsc --noEmit`, confirmed that Render compiles and runs the worker natively, and verified the complete live pipeline on Render without unnecessary local environment hacks."